

Project Report

TripWise Travel Planner & Budget Manager Course: Programming in Java (CSE2006)

Evaluation Type: Flipped Course Project Evaluation

Submitted by: Anshika Singh

Registration Number: 25BAI10764

Program: B. Tech Computer Science and Engineering (AI & ML)

Contents

1 Introduction 2 The Problem I Wanted to Solve 3 What the System Actually Does (Functional Requirements) 4 Under the Hood (Non-Functional Requirements) 5 System Architecture 6 Design Diagrams 6.1 Use Case Flow 6.2 Process Workflow 7 Why I Built It This Way (Design Decisions) 8 Implementation Details 9 Simulated Terminal Output 10 How I Tested It 11 The Bugs and Challenges I Faced 12 My Key Takeaways 13 Future Upgrades 14 References

1 Introduction

For my flipped course project, I developed the "TripWise" Travel Planner & Budget Manager. Instead of just writing a basic Java script, I wanted to build a fully functional, command-line application that brings theoretical Object-Oriented Programming (OOP) concepts to life. The system handles everything from destination cataloging and dynamic itineraries to automated budget calculations and capturing user expenses. By utilizing advanced Java capabilities like the Collections Framework and local File I/O, I aimed to create a travel planner that feels fast, non-blocking, and fault-tolerant, much like a real-world enterprise application.

2 The Problem I Wanted to Solve

When looking at traditional, manual trip planning, I noticed a few major bottlenecks that I wanted to fix with my code:

1. **Fragmented Information:** Keeping track of destinations in a notes app, dates on a calendar, and expenses in a spreadsheet makes planning chaotic.
2. **Human Error:** Calculating remaining budgets manually is prone to mistakes, and typos when entering values often crash poorly designed applications.
3. **Data Loss:** A lot of basic student projects lose all their data as soon as the terminal closes. I wanted my data to persist.

I built this project specifically to engineer a clean, exception-safe solution to these exact problems.

3 What the System Actually Does (Functional Requirements)

I designed the system around three core modules:

- **OOP-Driven Travel Management:** The system categorizes data into Destinations, Trips, and Activities, displaying them neatly for the user.
- **Dynamic Budget Engine:** A workflow that lets the user log individual expenses categorized by type (Food, Travel, Hotel), and automatically calculates the exact remaining budget.
- **Asynchronous Data Persistence:** The system takes the final ArrayLists and writes them to local CSV files in a `data/` folder.

4 Under the Hood (Non-Functional Requirements)

Beyond just making it work, I wanted the code to be efficient and safe. I focused on:

- **Performance:** By using Java's `ArrayList`, I made sure that managing dynamic arrays of destinations and expenses is fast and allows for infinite scaling without fixed bounds.
- **Reliability:** I hate when applications crash from a simple typo, so I built robust try-catch blocks and a custom validation class to handle bad inputs gracefully.
- **Data Safety:** I implemented File Handling to save all session data to `.csv` files locally.
- **Maintainability:** The code is strictly modular. I used OOP principles to separate the data models, the business logic, and the file I/O operations into different classes.

5 System Architecture

I split the application into a clean 3-layer architecture:

1. **User Interface Layer:** The terminal loop (`TripWise` main class) that takes commands and inputs from the user.
2. **Business Logic Layer:** The Manager classes (`TripManager`, `ExpenseManager`, etc.) which validate logic, do the math, and handle the dynamic arrays.
3. **Data Persistence Layer:** The `FileManager` class, a dedicated utility that formats and saves the data to the local disk.

6 Design Diagrams

6.1 Use Case Flow

- **Actor:** Traveler
- **Actions:** View Menu → Add Destination → Create Trip → Log Expense → (Handle Invalid Inputs if any) → Check Budget.
- **System Action:** Calculate Remaining Math → Write to CSV files.

6.2 Process Workflow

1. System boots and presents the Main Menu.
2. User types a numerical choice.
3. If the code is invalid, the system catches the exception and asks again.
4. If valid, the system asks for string/double details, validates them, and adds them to the `ArrayList`.
5. User types 5 to save data.
6. System calculates totals, passes a reference of the arrays to the `FileManager`, and successfully saves to disk.

7 Why I Built It This Way (Design Decisions)

- **Why an `ArrayList` over a standard array?** Standard arrays have a fixed length. Using an `ArrayList` lets the system dynamically grow as the user adds more trips or expenses.
- **Why use separate Manager classes?** I separated concerns so that if I decide to add a new feature (like a Packing List) next month, I can just add a new manager class without rewriting the core application loop.
- **Why CSV over binary files?** CSV files are easily readable by humans and programs like Excel, making the exported travel data highly portable.

8 Implementation Details

To keep things organized and meet the syllabus requirements, I structured the project using several distinct classes:

1. `Destination.java`, `Trip.java`, `Activity.java`, `Expense.java` (The Data Models)
2. `DestinationManager.java`, `TripManager.java`, `ActivityManager.java`, `ExpenseManager.java` (The Logic Handlers)
3. `Validator.java` (For custom error handling)
4. `FileManager.java` (To handle the I/O)
5. `TripWise.java` (The main loop where the program starts)

9 Simulated Terminal Output

```
*****
*           WELCOME TO TRIPWISE           *
*   Your Personal Travel & Budget Guide   *
*****
```

```
--- Main Menu ---
1. Manage Destinations
2. Manage Trips
3. Manage Itineraries (Activities)
4. Manage Expenses & Budget
5. Save All Data to Disk
6. Exit Application
What would you like to do? Enter choice: 4
```

```
>> Budget & Expenses Menu
```

```

1. Log a new expense
2. View all logged expenses
3. Check remaining budget
Select an option: 3
Enter Trip ID to check budget: 101
-----
Total Allocated Budget: ₹15000.0
Remaining Balance:      ₹3000.0
[STATUS] You are within your budget. Looking good!
-----

```

10 How I Tested It

- **Trying to break the inputs:** I intentionally typed garbage values like "XYZ123" for Trip IDs and words for budgets. My custom exceptions and `NumberFormatException` catch blocks successfully prevented the program from crashing.
- **Math Verification:** I ran several multi-item expenses through the system and manually calculated the totals to ensure the floating-point math was perfectly accurate.

11 The Bugs and Challenges I Faced

- **The Scanner Buffer Bug:** When I mixed `scanner.nextInt()` with `scanner.nextLine()`, the terminal kept skipping the prompt for the description because of a leftover newline character in the buffer. I fixed this by uniformly reading everything as a string using `scanner.nextLine()` and parsing the integers manually.
- **Directory Errors:** Initially, saving files threw an `IOException` if the `data/` folder didn't exist. I fixed this by utilizing `File.mkdirs()` to dynamically generate the required folders before writing the CSV files.

12 My Key Takeaways

This project was a huge step up from writing basic scripts. It forced me to actually apply theoretical concepts like Encapsulation and Modular Design to solve real problems. I also gained a much deeper understanding of how Java handles memory via the Collections Framework, and how tricky data persistence and file generation can be.

13 Future Upgrades

If I were to expand this project later, I would:

- Connect it to a real MySQL database using JDBC instead of saving to local CSV files.
- Build a Graphical User Interface (GUI) using JavaFX to make it look like a modern desktop planner application.

14 References

- Schildt, H. (2018). Java: The Complete Reference, 11th Edition. Oracle Press.
- Course Materials: CSE2006 - Programming in Java Syllabus & Modules.

- Oracle Official Java Documentation: Collections Framework and File I/O APIs.